How GitHub Sharded Their Monolithic Database

Source: Arpit Bhayani Dissection • GitHub Infrastructure
Focus: mysql1 Monolith, Virtual Partitioning & Schema Domains

1. The Monolith: Scaling `mysql1` to 950,000 QPS

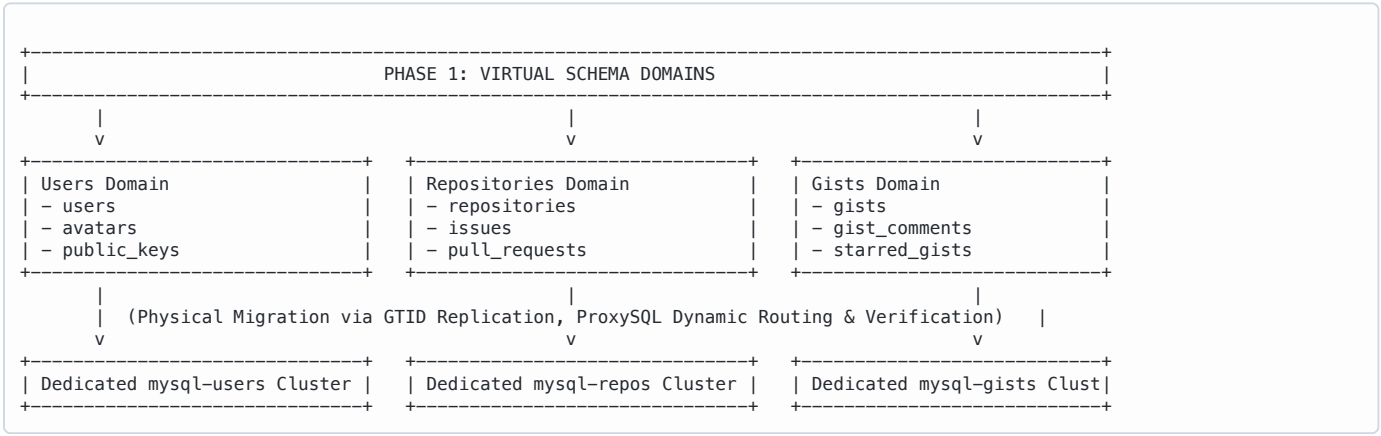
For over a decade, GitHub's flagship web application operated on a monolithic Ruby on Rails architecture backed by a single main MySQL cluster (`mysql1`). As the platform expanded to hundreds of millions of developers and repositories, `mysql1` faced extreme concurrency, peaking at over **950,000 queries per second (QPS)**.

Despite vertical scaling (128-core bare metal hosts, 2 TB RAM, NVMe RAID-0 arrays) and dozens of read replicas with ProxySQL, GitHub encountered the limits of shared storage:

- **Noisy Neighbor Contention:** High-frequency activity streams (notifications, audit logs) saturated buffer pool pages, degrading critical core flows like git push/pull and pull request reviews.
- **Uncontained Blast Radius:** Lock contention or schema locks on a secondary feature table had the power to cascade and knock down the entire GitHub platform.
- **Cross-Domain Sprawl:** Uncontrolled relational SQL `JOIN` queries spanned across distinct business domains, creating tight semantic coupling.

2. Phase 1: Virtual Partitioning via Schema Domains

Physical partitioning without preparation causes catastrophic failure: cross-database joins immediately fail. GitHub established a mandatory two-phase engineering paradigm:



> **The Virtual Partitioning Axiom:** Before moving any physical bytes, all logical boundaries must be established and enforced inside the application code. Tables are partitioned conceptually into *Schema Domains* in declarative YAML configuration files.

3. Enforcing Logical Boundaries with CI Linters

GitHub introduced automated Query and Transaction Linters into CI and local test suites:

- **Query Linter:** Intercepts ActiveRecord AST generation. Any query performing a `JOIN` or subquery across distinct schema domains fails the test suite immediately.
- **Application-Layer Hydration:** Engineers replaced SQL joins with multiple parallel point lookups, hydrating associations in Ruby application memory.
- **Transaction Linter:** Prevents multi-domain atomic transactions (`ActiveRecord::Base.transaction`), ensuring zero dependence on distributed Two-Phase Commit (2PC).

4. Zero-Downtime Physical Migration Topology

Once a schema domain was completely free of cross-domain queries and transactions, it was ready for physical extraction from `mysql1`:

```
Step 1: Clone cluster via Percona XtraBackup & GTID Replication
[ mysql1 Source Primary ] ==================(Continuous Async GTID Replication)================> [ New Target Primary ]
            |                                                                                          ^
            | (Read Traffic)                                                                           | (Shadow Read Mirroring)
            v                                                                                          |
+------------------------------------------------------------------------------------------------------------------------+
|                                          ProxySQL CONNECTION PROXY TIER                                                |
+------------------------------------------------------------------------------------------------------------------------+
            ^
            | (Queries)
[ GitHub Rails Application Fleet ]

Step 2: Sub-100ms Cutover
1. Application pauses writes for domain (< 50ms buffer in ProxySQL)
2. Verify target GTID replication lag == 0
3. ProxySQL swaps routing rules: domain writes/reads directed to New Target Primary
4. Release buffered writes; drop old tables from mysql1
```

5. Verification via Shadow Traffic & Checksumming

To ensure total zero-defect fidelity before cutover:

- **Shadow Read Mirroring:** ProxySQL asynchronously duplicated read traffic destined for the isolated domain to the target cluster.
- **Response Verification:** Background daemons compared result sets and row hashes between `mysql1` and the target cluster to confirm identical execution plans and indexes.
- **Replication Lag Guard:** Cutover was permitted only when GTID replication lag remained strictly below 1 second for 60 consecutive minutes.

6. The Sub-100ms Cutover Sequence

> **Cutover Mechanics:** ProxySQL briefly queued incoming write connections in its internal socket buffers for less than 50 milliseconds. Once replication caught up completely, ProxySQL hot-reloaded its routing table in memory, instantly steering new queries to the dedicated domain cluster. End users experienced zero downtime or failed requests.

Production Impact & Architectural Takeaways

7. Production Impact: 50% Primary Load Reduction

Partitioning `mysql1` into specialized schema clusters transformed GitHub's operational stability:

Metric / Dimension	Monolithic mysql1 (Before)	Partitioned Schema Domains (After)
Primary Database Load	Near 100% CPU & I/O saturation at peak	**50% reduction in CPU and disk I/O**
Incident Blast Radius	Global: Single table incident crashed site	**Strictly contained to individual schema cluster**
Replication Latency	Periodic replication stalls on heavy writes	Sub-millisecond GTID replication across clusters
Schema Migrations	High-risk global maintenance locks	Zero-downtime independent domain migrations (gh-ost)
Query Performance	Heavy multi-table joins saturating memory	Lean point-lookups with high cache hit ratios

8. Stepping Stone to Horizontal Sharding with Vitess

Functional partitioning by schema domain provided the clean foundation required for GitHub's subsequent migration to **Vitess**:

- Once domains (like Repositories) were physically isolated, they could be horizontally sharded across thousands of partitions using repository ID Vindexes.
- Without virtual schema partitioning, horizontal sharding would have been virtually impossible due to tangled multi-table joins.

9. First-Principles System Design Takeaways

- **Enforce Virtual Boundaries Before Physical Moves:** Never shard physical storage until application logic is strictly partitioned using automated linters in CI.
- **Application Joins Over Database Joins:** Breaking SQL joins into parallel application-level point lookups decouples database schemas and enables effortless cache hydration.
- **Connection Proxies Are Essential for Zero Downtime:** Intelligent proxy layers (ProxySQL) decouple applications from physical server topologies, enabling seamless sub-100ms cutovers.
- **Eliminate Distributed Transactions Early:** Designing schemas so each business write is contained within a single domain avoids the latency and fragility of Two-Phase Commit (2PC).